Development of a RAG-based Fantasy Book Recommender

Cosima Baumeier, 950619

November 27, 2025

1 Motivation and Goal

The goal of my project was to develop a Retrieval-Augmented Generation (RAG) system that provides personalized fantasy book recommendations. The idea is to allow users to describe what they are looking for in a fantasy novel. For example, the user might type *"I enjoyed the plot and setting of Harry Potter but I found the writing style too simple"* and the system should then suggest suitable books that match the user's preferences. The recommendation process is based on real public reader reviews from Goodreads, a social cataloging platform where users can rate books and write reviews. These reviews often contain rich qualitative information about the plot, writing style, characters, pacing, and overall impressions. Fantasy was chosen as the target genre because it is highly diverse, with a wide range of subgenres and themes. Reader preferences in fantasy tend to vary more strongly than in genres such as romance, where expectations are often more homogeneous.

2 Functionality of a RAG system

The idea of Retrieval Augmented Generation is to provide Large Language Models (LLMs) with contextual knowledge at inference time which is domain-specific. For that, a knowledge base is created by loading relevant contextual data from input sources, splitting the text into smaller chunks that can be handled by the LLM, using an embedding model to create a numerical (vector) representation of the meaning of each chunk and storing the embeddings in a vector database. When a user asks a question, the query is embedded using the same model as before. The chunks from the vector database that are most similar to this embedded question are retrieved. The LLM then answers the question based on the question and the provided context from the chunks. In my project, the book reviews form the knowledge base. The user states likes and dislikes in Fantasy books which is transformed into a preference profile by the LLM and gets embedded. Reviews which have a similar vector representation are used to recommend books.

3 Data Preparation

The data preparation was done in two parts: first, transforming the raw data from json into csv format and creating a subset of a manageable size, done in the *dataengineering.ipynb* notebook and described in sections 3.1 and 3.2. After that, the data had to be prepared to be suitable for the RAG development, done in the main notebook (*capstone.ipynb*) and described in 3.3.

3.1 The Dataset

The dataset was obtained from Goodreads Datasets[WM18, WMNM19]. The data was collected in late 2017 from goodreads.com by scraping user's public shelves. User IDs and review IDs are anonymized. I used the following three datasets:

- goodreads_reviews_fantasy_paranormal.json.gz: the review data
- goodreads_books_fantasy_paranormal.json.gz: metadata about books
- goodreads_book_authors.json.gz: metadata about authors

I created a Pandas dataframe by extracting the book id, the rating, and the review text from the review json file. I joined the book title from the book metadata file via the book id and the author name from the author metadata file via the author id. Only reviews containing Latin letters and those that have a minimum of 200 characters were included. This helped remove extremely short, low-information entries and ensure that the remaining reviews have enough content.

3.2 Creating a Subset

The data frame I created in step 3.1 contained more than 2 million reviews of more than 200,000 different books. Since embedding all those reviews for my RAG system would be costly and time-intensive, I decided to only keep a subset. I chose to include only books that have a minimum of 30 reviews in the dataset to remove very niche unpopular books. Also, I kept only 15 reviews per book to prevent redundant information about very popular books with thousands of reviews. I randomized the order of the reviews in case they were ordered in any way. After applying the filtering criteria, the final working corpus was reduced to 10,427 books and 156,405 reviews. An initial idea to reduce the dataset was to only keep positive reviews (four or five stars). I decided against that since negative and averagely rated reviews often state what the user didn't like in a book. From my personal experience, reviews with 2 to 4 stars tend to be much more nuanced than very positive or very negative reviews. When a user of my RAG writes a query like *"I enjoyed the plot and setting of Harry Potter but I found the writing style too simple"*, the retriever of my RAG might find a review that describes a magical world but complains about the *"way too sophisticated language"* which might be a good match for the user. On the other hand, when a review gets retrieved which describes a characteristic that the user stated as a dislike, the LLM in the final recommendation phase of the RAG graph can filter out that book. The *"rating"* column is therefore unnecessary and will not be used in any following steps.

3.3 Aggregating the reviews

Before the text could be split into chunks for the embedding, the individual reviews for each book had to be combined into a single, comprehensive text document. Before this step, I had a data frame with one row for every review. I now created a data frame with only one row per book. It has four columns: *"book_id"*, *"title"*, *"author"* and *"full_book_context"*. The *"full_book_context"* contains all 15 reviews for each book, separated by the delimiter *"- - -"*. This column is the one that gets embedded, the other columns serve as metadata.

4 RAG Development

For the RAG, I needed both an embedding model and a large language model. I could have downloaded models and run them locally on my computer. However, to achieve higher processing speeds, I decided to use OpenAI models and access them via the API. After a fellow student recommended it, I used OpenRouter, which allows multiple models to be run with only one API key. Primarily, this was done because I don't have a credit card which is needed for the OpenAI API.

4.1 Splitting Into Chunks

In order to prepare the aggregated book context for embedding and retrieval, the large text (all reviews for each book) first had to be broken down into manageable pieces, known as chunks. This chunking is vital for RAG systems as it ensures the vector database can perform efficient, semantically relevant searches and prevents the LLM from being overwhelmed by long context blocks. I used *RecursiveCharacterTextSplitter* from the Langchain library for the chunking. Each chunk was limited to a maximum of 500 characters (*chunk_size=500*), with an overlap of 50 characters (*chunk_overlap=50*) between consecutive chunks. The overlap ensures that critical context is not lost at the chunk boundaries. I defined a custom hierarchy of separators to minimize the disruption of meaning. The text splitter first tries to separate the chunks between each review (the *"- - -"* delimiter as explained in 3.3). If the chunk is still too big, it separates after a paragraph. After that, it separates after a line breaks. If it is still too large, then between words. The last resort is separating between characters. I defined a Python loop that iterates through every row of the aggregated dataframe and applies the text splitter to the *"full_book_context"* column and saves metadata (id, title and author) with each chunk.

4.2 Embedding

The resulting text chunks were embedded, meaning they were converted into vector representations. I chose the text-embedding-3-small model for this task, accessed via the OpenAI API base through OpenRouter. This model was chosen for its cost-effectiveness compared to larger models. Embedding one million tokens costs $0.02 which is very cheap. The embedding took more than an hour but had to be done only once. It is stored in a vector data base and doesn't have to be computed again.

4.3 Vector Data Base

ChromaDB was used to store the embedded chunks and their metadata. ChromaDB was initialized with the entire set of chunks and configured to persist the vector data base to a local directory. This vector store serves as the knowledge base.

4.4 Large Language Model

The openai/gpt-4o-mini model was chosen and accessed via the OpenAI API base through OpenRouter. It is a small and cheap model which still produces good results. The prices for one million tokens are as follows: $0.15 for input, $0.075 for cached input and $0.60 for output. One query and its answer from the final RAG system cost only a fraction of a cent.

4.5 Defining the RAG Graph

The RAG system is structured as a three-step pipeline: First, preferences are extracted from the user query. This preference profile is then used to retrieve reviews that have similar vector representations. The LLM then picks three to five book recommendations based on the preference profile and the given context (retrieved reviews and their metadata) and explains why they match. It's called a graph because the underlying LangGraph framework models the entire recommendation process as a series of connected nodes and edges. Each function is a node in the graph. In my case, I have three main nodes. The edges define the directional connection and flow of data between these nodes. The process must always proceed in a specific, ordered sequence. For example, the output of the *extract_preferences node* (the user's preference profile) becomes the input for the next node, *retrieve_books*.

4.5.1 Extracting preferences

The first step of my pipeline in the extraction of preferences based on the user query. I wrote a function that prompts the LLM to create a concise preference profile based on what the user liked, what the user dislikes, desired themes, tropes, tones, vibes, genres and specific elements they want more or less of. The output of the LLM is returned and used as input for the next step.

4.5.2 Retrieving books

The second step of my pipeline is the retrieval of books. I specified the retriever to check the vector database for the 20 most similar chunks based on the preference profile created in step 4.5.1. I wrote a Python loop that ensures that only one chunk per book gets retrieved by checking the book ids. If different chunks belong to the same book, only the most similar chunk is being kept as context for the next step. This is done to prevent context overload.

4.5.3 Generating recommendations

For the last step, I wrote a function that prompts the LLM to use the user query, the preference profile and the context (retrieved reviews with their metadata) to recommend 3-5 books. I instructed it to explain why the recommendation matches the user's preferences and to highlight specific themes, elements, or vibes. Also, to keep the tone friendly and helpful.

5 Streamlit App

I used Streamlit to create a pretty, user-friendly interface for my RAG system. I used *@st.cache_resource* to load the embedding and LLM. The vector data base is loaded from the local directory where it was persisted in step 4.3. The RAG Graph pipeline was copied from my notebook. I added a title, changed the background color, instructed the user with the text *"Explain what you like or dislike about Fantasy books. You could also mention your favorite books."* above the input field and implemented a loading spinner with the text *"Generating answer...be patient! :)"* to show the responsiveness since generating an answer takes 10-20 seconds. The app is hosted locally via accessing localhost. I considered hosting it online but found the process of pushing everything to GitHub (especially my large vector database) and figuring out how to safely implement the OpenAI API key too complicated.

Figure 1: Excerpt of the Streamlit app

6 Example results

6.1 Example 1 - specific like and dislike mentioned

I tried out the query *"I like books that, despite containing fantasy elements, remain realistic within our world. I don't like complicated names or family structures"*. It recommended five books. Here is the generated output for two of them:

> "The Night Circus" by Erin Morgenstern
> Why it matches: This novel offers a rich fantasy tableau without straying too far from a believable setting. The story revolves around a mysterious circus that appears only at night, bringing with it magical competitions between two young illusionists.
> Themes and elements: The characters are well-developed, and the narrative weaves elements of romance and friendship in a sense-of-wonder framework. The names, while unique, are not overly complicated, maintaining approachability.
>
> "The Ocean at the End of the Lane" by Neil Gaiman
> Why it matches: This book effectively blends fantasy and reality. It tells the story of a middle-aged man who returns to his hometown, triggering memories of a childhood encounter with a girl who introduced him to a world of magic.
> Themes and elements: The themes of memory, friendship, and the innocence of childhood resonate throughout, grounding the fantasy elements in emotional realism. Gaiman's straightforward prose makes it accessible, and the character relationships are uncomplicated.

6.2 Example 2 - specific book title mentioned

I tried out the query *"I like the setting and characters of Game of Thrones but I don't like how brutal it is"*. It recommended five books. Here is the generated output for two of them:

> "The Lies of Locke Lamora" by Scott Lynch
> Why it matches: This book features a richly built world and a cast of complex, charismatic characters, much like Game of Thrones but with a more light-hearted and whimsical approach. The brutality and graphic violence are significantly toned down.
> Themes and elements: The story revolves around a group of con artists in a fantastical version of Venice. You'll find themes of friendship, clever schemes, and a detailed heist plot that provides plenty of adventure without excessive violence.
>
> "The Goblin Emperor" by Katherine Addison
> Why it matches: This standalone novel focuses heavily on character development, political intrigue, and a deeply immersive world. It avoids typical fantasy tropes, particularly violence, making it a refreshing read.
> Themes and elements: It follows Maia, a half-goblin who unexpectedly becomes emperor. The story delves into themes of ethics, honor, and the struggle for acceptance in a court full of intrigue. The character relationships and court politics are rich and engaging.

7 Limitations

Although the RAG system works and seems to give good recommendations, it has some limitations. One very obvious limitation is that the data was collected in late 2017 which makes the dataset eight years old. Consequently, the system is unable to recommend books published after this cutoff date, making the recommendations outdated and potentially missing modern fantasy trends. Secondly, the data preparation involved a filtering step, selecting only books with 30 or more reviews. While this was necessary to reduce the number of reviews to a manageable amount, it inherently biases the system toward popular books. Third, the language of the source review texts could not be verified during data preparation. Although the dataset is assumed to be predominantly English, reviews in other languages could potentially be included. Since the embedding model is made for English language,

this would be noise and can negatively impact the semantic accuracy of the vector representation and therefore also the final recommendations. Furthermore, the entire corpus of reviews, being sourced from Goodreads, is subject to social bias common on public review platforms. Reviews often display inflation (a tendency towards higher ratings) and herding behavior, meaning the aggregated context may not reflect truly neutral or critical opinions, but rather group consensus. I included a deduplication step in the retrieval process which ensures only the single most relevant chunk per book is passed to the LLM. This can discard valuable information. Specifically, a user's dislikes (e.g. no brutality) might be described in a less relevant chunk that is then filtered out, while the chosen chunk focuses only on the book's positive aspects. The book then gets recommended despite being unsuitable.

References

[WM18] Mengting Wan and Julian J. McAuley. Item recommendation on monotonic behavior chains. In Sole Pera, Michael D. Ekstrand, Xavier Amatriain, and John O'Donovan, editors, *Proceedings of the 12th ACM Conference on Recommender Systems, RecSys 2018, Vancouver, BC, Canada, October 2-7, 2018*, pages 86–94. ACM, 2018.

[WMNM19] Mengting Wan, Rishabh Misra, Ndapa Nakashole, and Julian J. McAuley. Fine-grained spoiler detection from large-scale review corpora. In Anna Korhonen, David R. Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 2605–2610. Association for Computational Linguistics, 2019.